

Denial Prediction

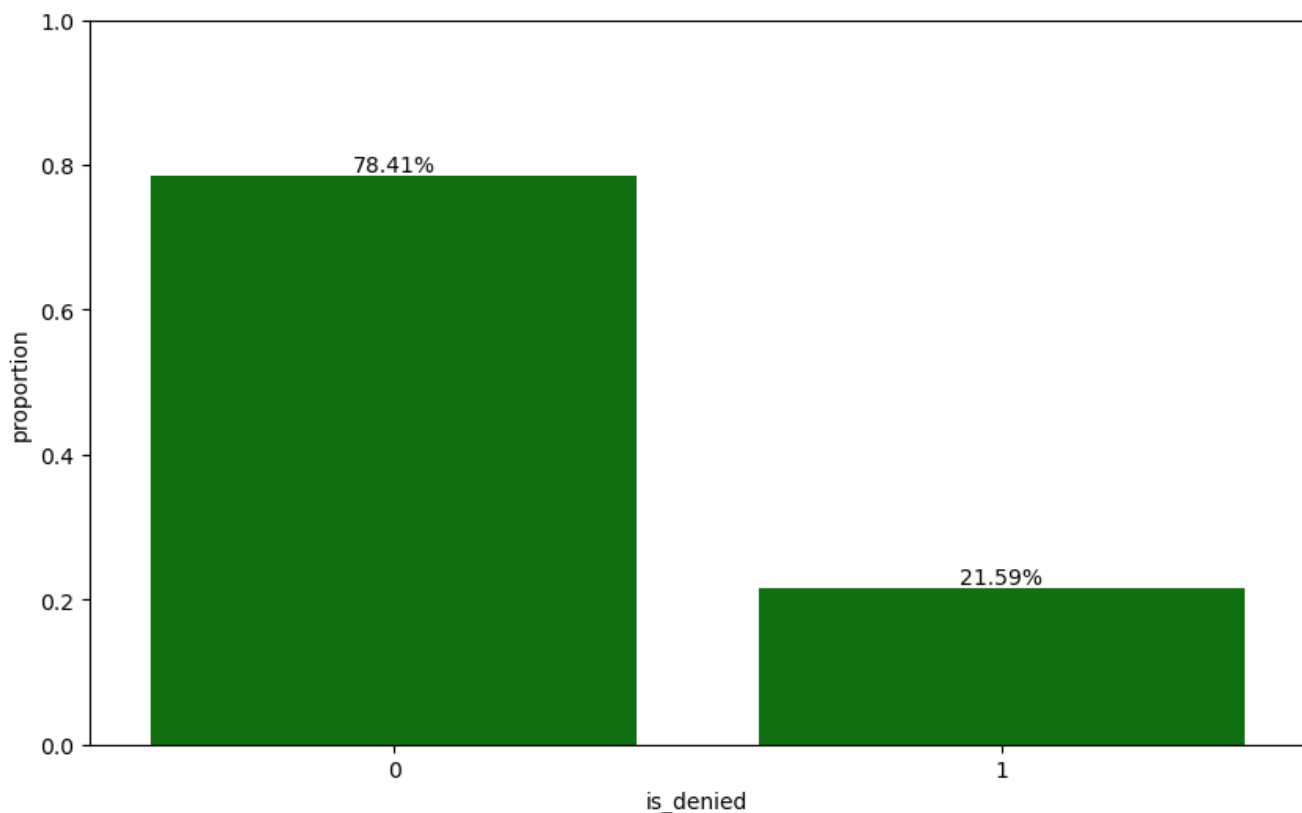
How you framed the problem — which errors matter more and why.

The one line in the brief that shaped everything: the review team can only inspect the top 25% of claims by risk before they go out. That turns this from a plain yes/no classifier into a triage problem. Generated model's real job is to sort the day's claims so the ones most likely to be denied float to the top of a queue the team is going to work through anyway — getting the ordering right matters far more than any single accept/reject decision.

Not all mistakes cost the same.

- A missed denial — a claim we let through that comes back denied — is the expensive error: the hospital has to investigate it, correct it, resubmit, and wait weeks to get paid.
- A false alarm (flagging a clean claim) only costs a couple of minutes of an analyst's time, and because the team reviews a fixed 25% either way, a false alarm doesn't even add work — it just takes up a review slot. So I optimized for catching as many real denials as possible inside that 25% budget, and treated overall accuracy as secondary.

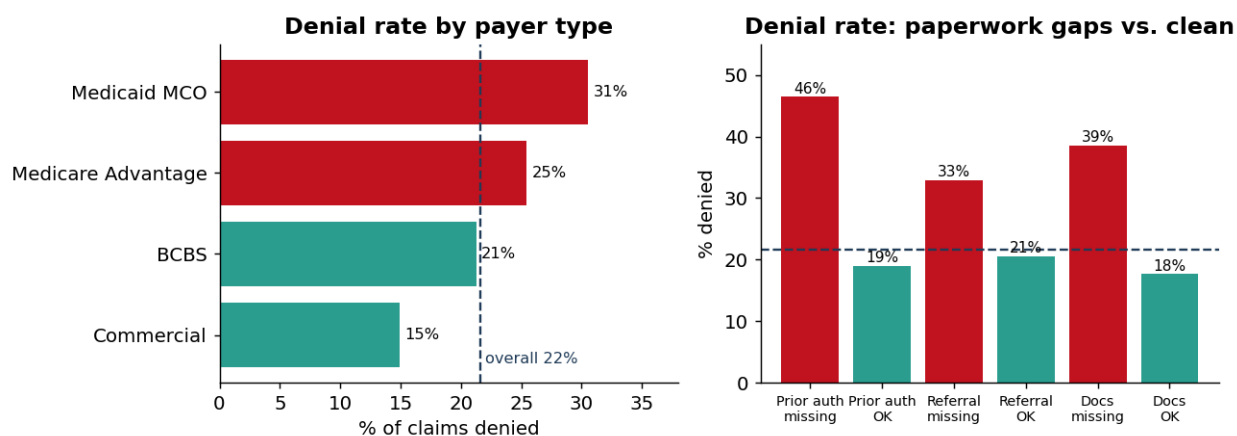
Target distribution suggests about 22% of claims are denied.



Distribution of Target "is_denied" with proportion

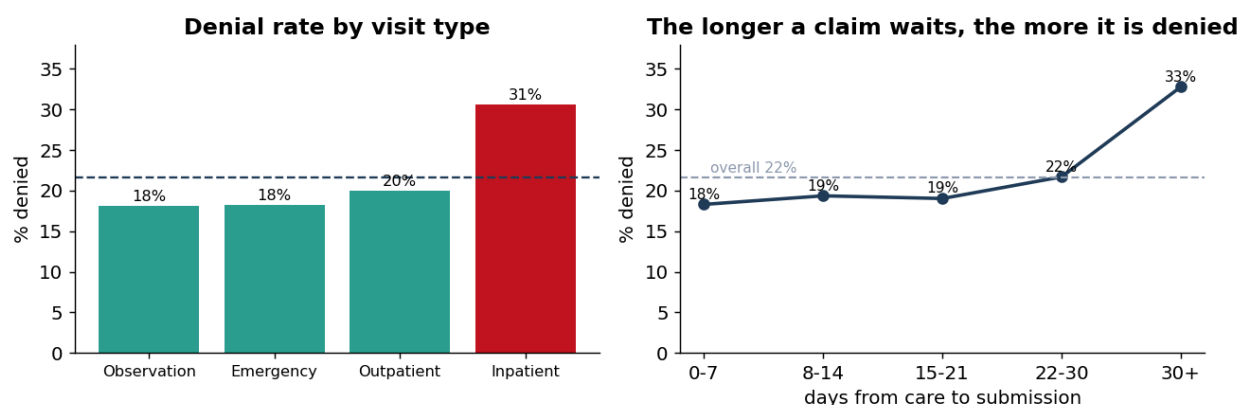
Two or three data findings worth sharing with a non-technical manager

Three things jumped out of the history that a manager can act on without any modeling background:



Left: denial rate by payer type against the 22% overall rate. Right: claims with a paperwork gap are denied far more often than clean ones.

- Who the payer is tells you a lot. Medicaid MCO claims are denied about twice as often as Commercial ones (31% vs 15%). Before we look at anything else on the claim, the payer already shifts the odds.
- Missing paperwork is the clearest warning sign. When a prior authorization is required but isn't on file, the denial rate jumps to 46% — more than double the 22% baseline. Missing documentation (39%) and missing referrals (33%) tell the same story. These are exactly the gaps a front-end team can close before the claim ever goes out.
- Sitting on a claim hurts, and inpatient stays are riskiest. Claims submitted more than 30 days after care are denied 33% of the time, and inpatient visits are denied at 31% vs ~18–20% for everything else.



Left: inpatient claims are denied far more often than other visit types. Right: the longer a claim waits between care and submission, the more likely it is denied.

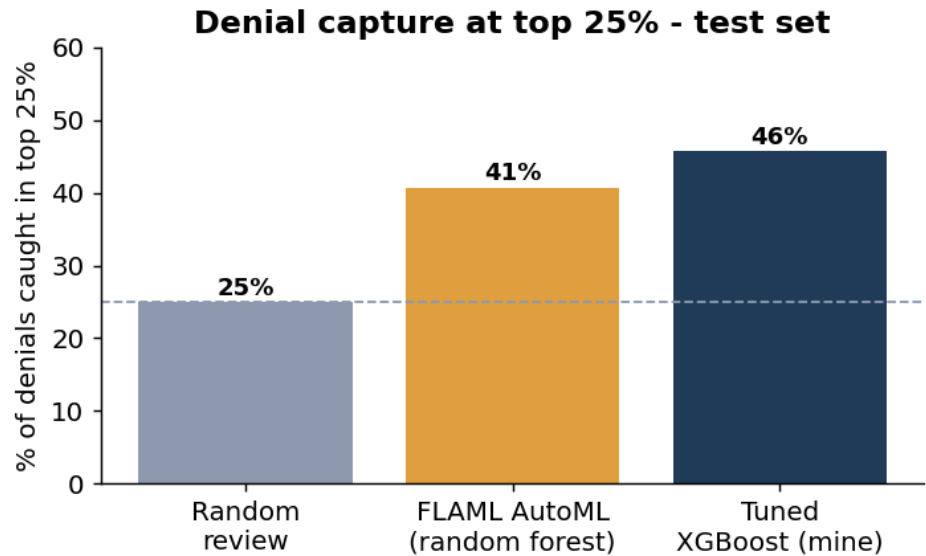
What I built, what baselines I compared against, and my threshold choice

Features. I kept it honest on leakage: `is_denied` and `denial_reason` are never used as inputs (`denial_reason` only exists after a denial has already happened). On top of the raw fields I added four features that encode the real risk pattern — `prior_auth_gap` and `referral_gap` (“required but not on file”), `payment_ratio` (expected / billed), and a log of the billed amount to tame its long tail. The exact same engineering runs on the current claims so they line up column-for-column.

Baselines, in order.

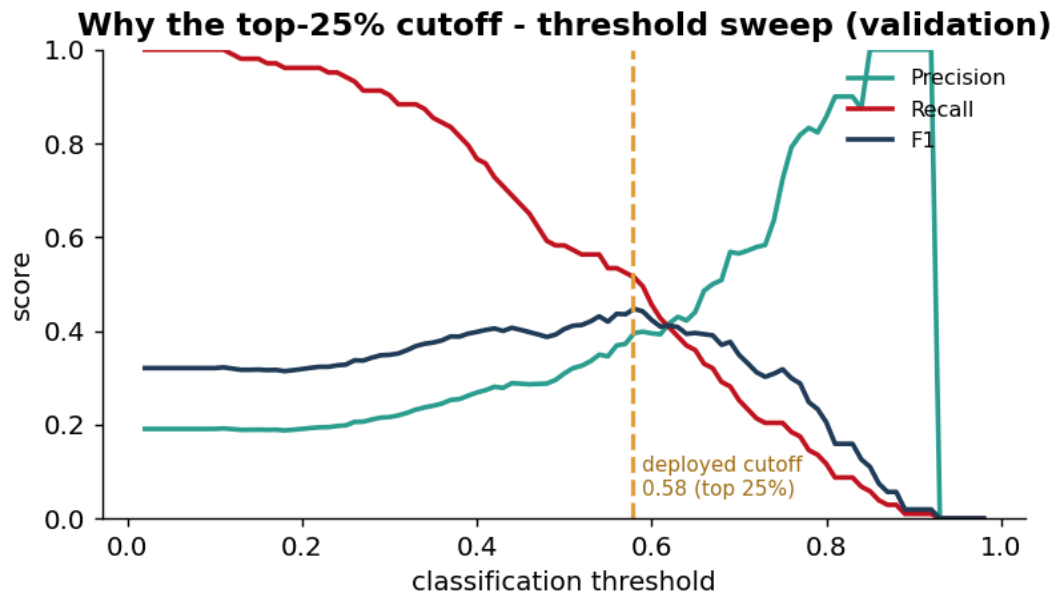
- (1) Reviewing claims at random catches ~25% of denials in the top 25% — the floor any model has to beat.

(2) A one-minute FLAML AutoML search landed on a small random forest and got to 41%; a real jump, but its numbers bounced around between validation and test. (3) A tuned XGBoost — an 80-config randomized search, early-stopped directly on denial-capture-at-25% — reached 46% and was the steadiest, so that's what I shipped.



Each step over the random-review floor. The tuned XGBoost catches 46% of denials in the top 25% reviewed — about 1.8x the random baseline.

Threshold. I set the cutoff at the 75th percentile of the validation scores (0.58), which flags exactly the top 25% of claims by volume — so the “flagged” set matches what the team can physically review, no more and no less. I did look at the F1-optimal and Youden's-J cutoffs, but capacity is the constraint that matters here, so the capacity cutoff is the one I deployed. On the 500 current claims it flags 115 as High risk (23%) — right on budget. Risk tiers follow the same logic: High above the 25% cutoff, Medium down to the median score, Low below.



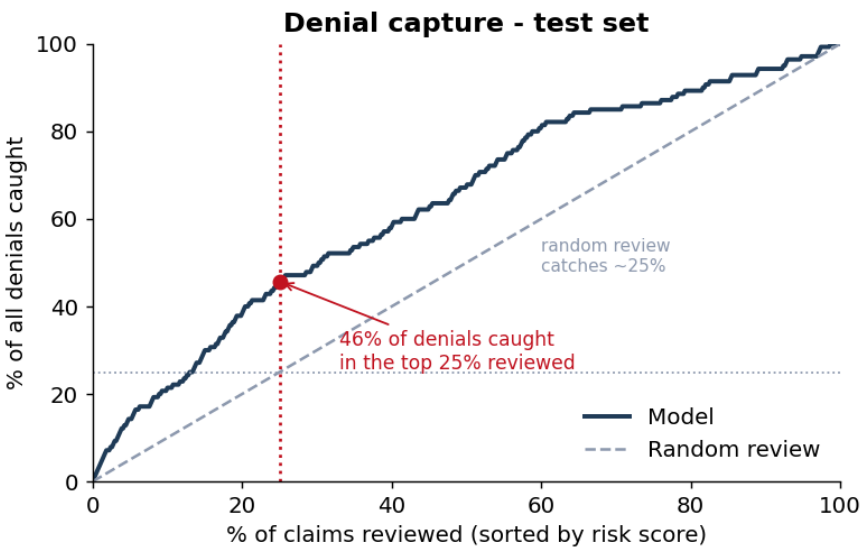
Precision, recall and F1 as the threshold moves (validation). The deployed cutoff (amber) sits at the top-25% capacity point rather than at the F1 peak — a deliberate, constraint-driven choice.

Test set metrics and denial capture at top 25%

I held the test split back until the very end and tuned nothing on it. The headline: reviewing the top 25% of claims by score, the team catches about 46% of all denials — roughly 1.8x what they'd catch reviewing 25% at random, and nearly half of every denial while touching only a quarter of the volume.

Test-set metric	Value
Denial capture @ top 25% (primary)	0.457
Precision @ top 25%	0.474
PR-AUC (base rate 0.26)	0.484
ROC-AUC	0.684
At the deployed threshold: precision / recall / F1	0.46 / 0.47 / 0.47

Reading it in plain terms: nearly half of what an analyst opens in that top slice is a genuine denial (47% precision, against a 26% base rate), and the team reaches almost half of all denials for a quarter of the effort. It's a modest but real lift — and an honest one for synthetic data that carries a lot of noise no model can remove.



Denial-capture curve. The model rises well above the random-review diagonal; the red point is the operating point (46% of denials caught within the top 25%).

Test-set decisions at the cutoff

Actually not denied	322 True negative	77 False positive
	74 Missed denial	66 Caught denial
Actually denied	Predicted not denied	Predicted denied

Your LLM prompt, one example explanation, and one honest limitation

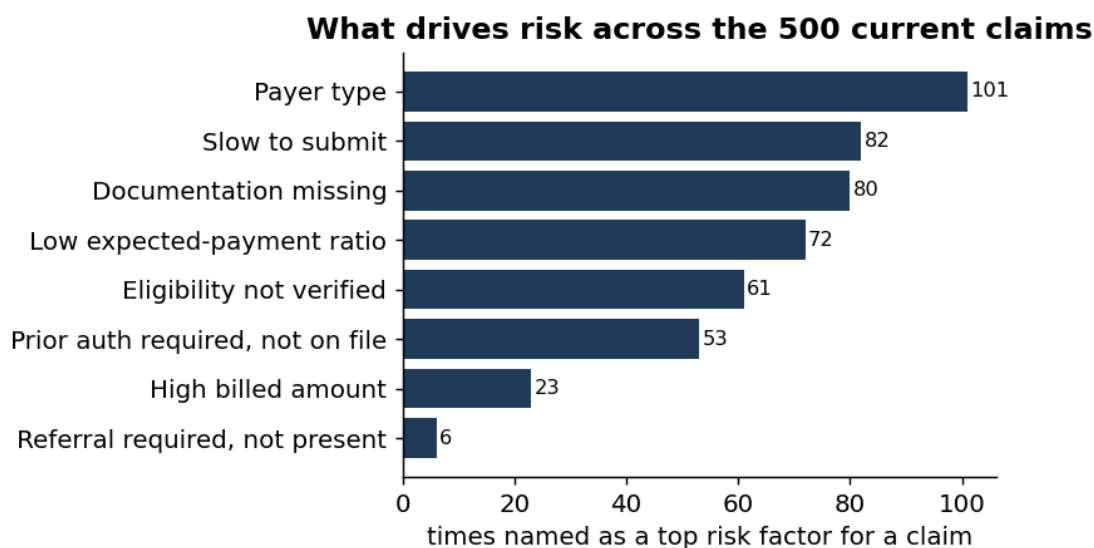
For each risky claim I first pull the model's own reasons using SHAP (which fields pushed this specific claim's score up or down), then hand only those facts to the LLM. The prompt is built so the model can't wander:

Prompt:

System: *You are a claims-denial analyst assistant. Write a short, plain-English risk explanation for ONE claim. Use ONLY the facts provided (probability, tier, risk drivers) — do not invent codes, names, or dollar amounts. Plain language, no jargon. Exactly 2-3 sentences. Include exactly ONE concrete recommended action. Make clear this is a risk ESTIMATE, not a guarantee.*

User: *the claim's probability, risk tier, and its SHAP drivers — nothing else.*

The chart below shows why grounding in SHAP matters — the same handful of drivers (missing prior auth, missing documentation, unverified eligibility, low payment ratio, slow submission) account for almost every High-risk claim, so the explanations stay concrete and repeatable.



The risk drivers the model most often flags across the 500 current claims — these are what the LLM turns into plain-English guidance.

Example — CCLM-00285, the single riskiest current claim (86%):

System: *You are a claims-denial analyst assistant. Write a short, plain-English risk explanation for ONE claim. Use ONLY the facts provided (probability, tier, risk drivers) — do not invent codes, names, or dollar amounts. Plain language, no jargon. Exactly 2-3 sentences. Include exactly ONE concrete recommended action. Make clear this is a risk ESTIMATE, not a guarantee.*

User: *the claim's probability, risk tier, and its SHAP drivers — nothing else.*

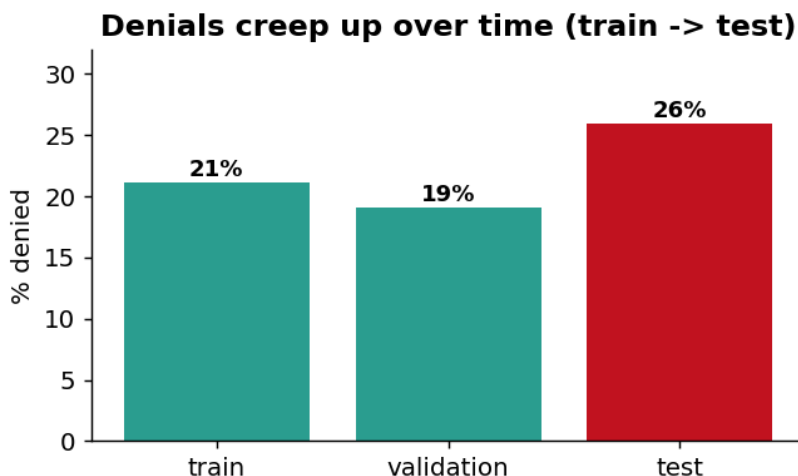
Low-risk sanity check — CCLM-00208 (8%): “The estimated risk of denial for this claim is low, with an 8% probability based on the total billed amount, payer type, and payment ratio. To further reduce the risk, ensure all documentation is complete and accurate before submission. Remember, this is a model risk estimate and not a guarantee of denial.” It stays calm, recommends only a routine check, and still hedges — it doesn't cry wolf on a clean claim.

One honest limitation. The explanation is only as trustworthy as the drivers behind it. SHAP tells us which fields moved the score, not that fixing them will stop the denial — these are correlations learned on synthetic data. The

LLM never invents facts, but it does sometimes soften a numeric driver (“low payment ratio”) into vaguer language and can sound a touch more confident than a borderline 55% score deserves. I'd want to measure that, not just eyeball it.

What I would improve with more time

- Calibrate the probabilities (isotonic or Platt) so “86%” actually means 86%. Right now the model ranks well but I haven't verified the numbers are calibrated.
- Bring in cost. A \$70k inpatient denial and a \$1k one aren't equal. Weighting by expected_payment would rank claims by dollars at risk, which is what the business cares about.
- Chase the drift shown below. The test slice is denied more often (26%) than train (21%). Before trusting this in production I'd find out whether that's time, payer mix, or something else — and backtest over time instead of on one fixed split.



- Add signal and a real eval. Payer-level historical denial rates as a feature, plus a small scored eval set for the explanations so I can catch the under-explaining problem by measurement rather than by reading them one at a time.